

MAILTRACE AI

36-HOUR WORKING PROTOTYPE PLAN

SIH26106 | Team Echoshield | Blockchain & Cybersecurity

AI-Powered Email Threat Detection & Forensic Intelligence Platform

Purpose: turn the selected SIH idea into a reliable, demo-ready MVP in 36 hours, starting from a clean Windows development machine and using Google Colab or Kaggle for model training because the local machine has no GPU.

Core investigation story: Detect -> Verify -> Trace -> Correlate -> Explain -> Report

Primary MVP principle: Build the smallest end-to-end system that works on real .eml files. External threat-intelligence services and Transformer inference are enrichment layers, not single points of failure.

1. What the 36-hour MVP must do

- Accept a raw .eml file or load one of three prepared demo emails.
- Preserve the original message and compute a SHA-256 evidence hash.
- Parse MIME, headers, Received chain, sender identities, URLs, domains, IPs and attachments.
- Verify SPF, DKIM and DMARC where evidence is available; separately evaluate alignment and identity consistency.
- Run phishing/BEC detection using a reliable local model/rules path.
- Enrich public IP/domain infrastructure with DNS, RDAP and GeoIP when available.
- Fuse signals into a 0-100 explainable risk score with evidence contributions.
- Show timeline, observed route, infrastructure map and relationship/campaign graph.
- Generate an evidence-backed PDF and JSON forensic report.
- Continue core analysis when external APIs are unavailable.

2. Frozen technology stack

Versions below were checked against official release/package sources on 14 September 2026. Patch versions for some frontend libraries are intentionally not hard-coded here; after installation, package-lock.json and the Python lock/requirements file become the exact reproducible record.

Area	Choice	Role
OS / shell	Windows 10/11 + PowerShell	Local development target
Python	3.13.15	Backend + forensics + ML fallback
Node.js	24.21.0 LTS	Frontend toolchain
React	19.3	Frontend
Build	Vite + TypeScript	Frontend build/dev server
UI	Tailwind CSS	Minimal dashboard UI
Backend	FastAPI 0.141.1 + Uvicorn	REST API
Validation	Pydantic	Typed API schemas
ORM	SQLAlchemy 2.x	Database abstraction
AI runtime	PyTorch 2.14	Training/inference

NLP	Transformers 5.17.0	Optional fine-tuned Transformer
ML fallback	scikit-learn	TF-IDF + Logistic Regression
Mail forensics	Python email + mail-parser 4.6.5	MIME/header parsing
Auth/DNS	dkimpy + dnspython	DKIM + DNS/SPF/DMARC support
Domain parsing	tlldextract	Registered-domain handling
GeoIP	geoip2 5.3.0 + GeoLite2	Observed infrastructure geolocation
Threat intel	RDAP + DNS + optional reputation APIs	Enrichment
Graph	NetworkX + Cytoscape.js	MVP correlation/visualization
Map	Leaflet	Infrastructure map
Charts	Recharts	Risk/evidence visuals
Database	PostgreSQL 18.6	Case data + metadata
Evidence	Local evidence store + SHA-256	Originals + integrity
Reports	Jinja2 + HTML/CSS + PDF renderer	PDF + JSON report
Deployment	Docker Compose	Reproducible services
Training GPU	Google Colab or Kaggle	Remote training only

3. Compatibility decisions and what NOT to depend on

1. Keep Python 3.13 for the hackathon instead of moving the whole stack to Python 3.14. Python 3.13.15 is maintained and has mature package support.
2. Use Node 24 LTS, not the current Node 26 feature line, for a stable frontend environment.
3. Use React 19.3; it is the current React release.
4. Use PyTorch 2.14 and Transformers 5.17.0 in the training environment. Train remotely on Colab/Kaggle; export the resulting model artifact to local storage.
5. Use a classical local inference fallback (TF-IDF + Logistic Regression). This guarantees the app remains demoable even if a Transformer model is slow or cannot load.
6. Use NetworkX in the backend and Cytoscape.js in the browser. Do not introduce Neo4j during the critical 36-hour build.
7. Use PostgreSQL 18.6 for the final prototype, but start development with SQLite if database setup begins to block progress. SQLAlchemy allows the swap.
8. Treat RDAP, DNS, GeoIP and reputation providers as optional enrichment. Every network call gets a timeout and a safe unavailable state.
9. Do not make Gmail/Microsoft OAuth, Kafka, Kubernetes, Redis, a live SMTP gateway, or blockchain a hard dependency for the MVP.

4. Machine setup - Step 0 before coding

Run these commands on the machine that will host the frontend/backend. This is the first checkpoint; do not install the rest of the stack blindly.

```
python --version
py --version
node --version
npm --version
git --version
docker --version
docker compose version
nvidia-smi
where python
where node
where git
```

Expected local GPU result: No NVIDIA GPU is required. If nvidia-smi is missing, that is acceptable for this plan. Training happens in Colab/Kaggle.

5. Install on Windows

5.1 Install the base tools

- Git for version control.
- Python 3.13.15. During install, enable "Add Python to PATH".
- Node.js 24.21.0 LTS.
- Docker Desktop only if the team will use Compose locally. If Docker setup itself becomes a blocker, run the app directly first and containerize after integration works.
- VS Code + Python + ESLint/TypeScript extensions.

5.2 Create the repository and environments

```
mkdir MAILTRACE-AI
cd MAILTRACE-AI
git init
git branch -M main

py -3.13 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

5.3 Backend installation

```
mkdir backend
cd backend
python -m pip install fastapi==0.141.1 uvicorn[standard] pydantic sqlalchemy
python -m pip install mail-parser==4.6.5 dkimpy dnspython tldextract geoip2==5.3.0
python -m pip install scikit-learn pandas numpy requests httpx jinja2
cd ..
```

5.4 Frontend installation

```
npm create vite@latest frontend -- --template react-ts
cd frontend
npm install
npm install axios react-router-dom cytoscape leaflet react-leaflet recharts lucide-react
npm install -D tailwindcss postcss autoprefixer
cd ..
```

Compatibility checkpoint: Start the frontend with `npm run dev` and the backend with `uvicorn app.main:app --reload`. Confirm the browser can call `GET /api/health` before adding business logic.

5.5 Database

Preferred final environment: PostgreSQL 18.6. For the first 2-4 hours, SQLite is acceptable to remove installation friction; then migrate the same SQLAlchemy models to PostgreSQL once the API contract is stable.

```
# Final DB target
DATABASE_URL=postgresql+psycopg://mailtrace:password@localhost:5432/mailtrace
```

Install a PostgreSQL driver such as psycopg during the database phase, not before the parser is working.

6. No local GPU: training strategy

The local machine is a deployment/integration machine, not the model-training machine. Use Google Colab or Kaggle for fine-tuning, evaluation and model export. The prototype should still work locally with the classical ML fallback if the Transformer cannot be loaded.

6.1 Training workflow

```
Local data / public dataset
  |
  v
Google Colab or Kaggle
  |
  +--> train / validate
  +--> evaluate
  +--> save model + tokenizer
  v
Download artifact
  |
  v
MAILTRACE-AI/ml/models/
  |
  v
Local FastAPI inference
```

6.2 Model plan

- Baseline first: TF-IDF on subject + body + selected header-derived text, followed by Logistic Regression. This should run locally on CPU and become the guaranteed fallback.
- Then fine-tune a small Transformer such as DistilBERT in Colab/Kaggle if GPU quota/time allows. Keep sequence length moderate and use early stopping.
- Export the tokenizer and model directory together. Test the artifact in a clean CPU-only local virtual environment.
- If Transformer CPU latency is too high for the judge demo, use the classical model for the live score and show the Transformer as the trained semantic component in the research/evaluation section. Do not fake inference.

6.3 Colab starter commands

```
!pip install -U torch transformers datasets evaluate accelerate scikit-learn pandas numpy
```

```
# Example local artifact export
from transformers import AutoTokenizer, AutoModelForSequenceClassification
model.save_pretrained("mailtrace-email-model")
tokenizer.save_pretrained("mailtrace-email-model")
```

6.4 Dataset / evaluation structure

File	Required fields	Purpose
train.csv	text,label	phishing/BEC/legitimate or binary malicious/benign
val.csv	text,label	model selection
test.csv	text,label	final holdout evaluation

Evaluation rule: Keep a fixed untouched test set. Record precision, recall, F1, ROC-AUC and a confusion matrix. Do not report a single accuracy number without the class distribution.

7. Backend architecture and data contract

```
POST /api/cases/upload
-> preserve raw bytes + SHA-256
-> parse .eml
-> extract headers/MIME/IOCs
-> verification
-> AI + rules
-> intelligence enrichment
-> evidence fusion
-> graph/timeline
-> report
```

7.1 Required API endpoints

Method	Path	Purpose
GET	/api/health	service check
POST	/api/cases/upload	create case + preserve .eml
POST	/api/cases/{id}/verify	SPF/DKIM/DMARC + identity checks
POST	/api/cases/{id}/analyze	full evidence fusion
GET	/api/cases/{id}	summary
GET	/api/cases/{id}/headers	raw/parsed headers
GET	/api/cases/{id}/graph	nodes + edges
GET	/api/cases/{id}/timeline	observed routing timeline
GET	/api/cases/{id}/report	PDF/JSON report

7.2 Canonical result JSON

```
{
  "case_id": "MT-2026-001",
  "classification": "CRITICAL",
  "risk_score": 91,
  "confidence": 0.94,
  "scores": {
    "ai_threat": 88,
    "authentication": 72,
    "identity": 91,
    "url": 86,
    "infrastructure": 68,
    "bec_behavior": 93
  },
  "reasons": [
    "Reply-To mismatch",
    "Lookalike domain",
    "Urgent payment request"
  ]
}
```

8. Forensic pipeline - build in this order

Stage	Output	Rule
1. Preserve	raw .eml + SHA-256	No analysis before evidence is stored
2. Parse	MIME, headers, body, attachments	Use Python email + mail-parser
3. Extract	URLs, domains, IPs, hashes	Normalize and deduplicate
4. Verify	SPF/DKIM/DMARC + alignment + identity	Return PASS/FAIL/UNKNOWN
5. Detect	rules + ML	Independent evidence signals
6. Enrich	DNS + RDAP + GeoIP	Timeout + fallback
7. Fuse	weighted evidence score	Explain every major contribution
8. Correlate	graph + campaign similarity	Shared IP/domain/URL/hash/etc.
9. Report	PDF + JSON + audit trail	Investigator-ready output

9. Risk scoring - safe implementation

AI / NLP	0-25
Identity mismatch	0-20
Authentication	0-15
URL / Domain	0-15
Infrastructure	0-15
Campaign	0-10

TOTAL	0-100

- Make the weights configurable.
- Do not claim the weights are scientifically optimal unless you evaluate them.
- Store each contribution so the judge can inspect why the risk changed.
- Keep confidence separate from risk. A high risk score with weak evidence should not be presented as high confidence.

10. Frontend: one investigation dashboard

#	Component	User action
1	Inbox / Upload	Select demo or upload .eml
2	Email viewer	Visible sender, subject, body, suspicious cues
3	Verify Email	SPF/DKIM/DMARC, Reply-To, Return-Path, identity consistency
4	Full AI Scan	Seven-stage pipeline progress
5	Risk result	0-100 score + classification
6	Score matrix	AI, Auth, Identity, URL, Infrastructure, BEC, Confidence
7	Explainable AI	Evidence contributions + decision path
8	Raw Headers	Underlying technical evidence
9	Mail Route	Observed source -> relay -> recipient
10	Map	Observed infrastructure geolocation
11	Graph	Email -> domain -> URL -> IP -> ASN -> campaign
12	Report	PDF + JSON

11. Campaign graph model

Nodes: Email, Sender, Domain, Subdomain, IP, ASN, URL, Attachment Hash, DKIM Selector, Mail Server, Nameserver, Campaign, Victim. Edges: SENT_FROM, CLAIMS_IDENTITY, REPLIES_TO, CONTAINS, RESOLVES_TO, USES, BELONGS_TO, HAS_HASH, SIMILAR_TO, PART_OF.

Correlation signals: shared IP/ASN, shared nameserver, shared URL/domain, shared attachment hash, shared DKIM selector, and semantic similarity.

12. Exact 36-hour execution plan

Hours	Work	Exit condition
0-1	Machine audit + repo	Verify Python/Node/Git/Docker; create branches; assign owners
1-3	Environment	Create venv; install backend/frontend; health endpoint; React-FastAPI connection
3-7	Email parser	MIME, headers, Received chain,

		URLs/IPs/domains/attachments; first .eml passes
7-10	Verification	SPF/DKIM/DMARC, alignment, From/Reply-To/Return-Path
10-13	Detection baseline	Rules + TF-IDF/LogReg; output reasons and score components
13-16	Evidence fusion	Final 0-100 risk; confidence; explanation JSON
16-20	Threat intel	DNS/RDAP/GeoIP with timeout/fallback; infrastructure result
20-23	Database	Cases, emails, indicators, analysis, audit events
23-26	Graph + timeline	NetworkX graph JSON, Cytoscape view, observed route
26-29	Frontend integration	Dashboard, verify, scan, result, XAI, headers, map
29-31	Colab/Kaggle model	Fine-tune optional Transformer; export artifact; CPU smoke test locally
31-33	Report + evidence	PDF, JSON, SHA-256, audit trail
33-34	Three demo cases	BEC, credential phishing, legitimate
34-36	Freeze + stress test	No new features; run end-to-end demo 5+ times; prepare fallback video/screenshots

13. Team division for 4 members

Member	Owner area	Deliverables
Member 1	Backend + Email Forensics	FastAPI, parser, auth, cases, evidence
Member 2	AI/ML	dataset, baseline, Colab/Kaggle Transformer, scoring
Member 3	Frontend	React dashboard, XAI panel, graph, map
Member 4	Intel + Integration	DNS, RDAP, GeoIP, DB, report, integration tests

Coordination rule: Agree on the canonical JSON response before parallel coding. Everyone integrates against that contract instead of changing field names ad hoc.

14. Failure testing checklist

Failure	Expected behaviour
Malformed .eml	Graceful parse error; case marked invalid
Missing Reply-To	No crash; mismatch check becomes false/unknown
Missing Auth results	Return UNKNOWN; continue
Private IP	Do not geolocate as public infrastructure
No public IP	Map section says no public source IP observed
DNS timeout	Continue with unavailable flag
RDAP timeout	Continue with unavailable flag
GeoIP unavailable	Continue with location unknown
Transformer unavailable	Use local classical model fallback
External API rate-limited	Use cache / skip provider
Duplicate email	Hash detects same raw evidence
PDF failure	JSON report still available
DB failure	Clear error and preserve raw evidence before persistence when possible

15. Judge-safe forensic language

Avoid	Preferred wording
Do not say	Say instead
"We found the attacker location."	"We identified the observed source infrastructure and its geolocation with confidence."
"The IP proves the attacker."	"The IP is one piece of infrastructure evidence; it does not establish human identity."
"Authenticated means safe."	"Authentication is one signal; authentication success alone does not establish legitimacy."
"The AI knows it is malicious."	"The AI and forensic signals produce an explainable risk assessment."
"We traced the attacker."	"We reconstructed observed routing and correlated infrastructure evidence."

16. Features to postpone

- Full Gmail/Microsoft OAuth integration
- Live SMTP gateway / mail proxy
- Kafka, Kubernetes, Elasticsearch, full SIEM ingestion
- Neo4j production deployment
- Custom LLM pretraining
- Blockchain-based evidence ledger
- Mobile app
- Exact human attribution from IP geolocation

17. Definition of done

A judge can open MAILTRACE AI

- > choose BEC demo
- > click Verify Email
- > see SPF/DKIM/DMARC + identity evidence
- > click Full AI Scan
- > see the analysis stages
- > get a risk score + confidence
- > inspect evidence contributions
- > open raw headers
- > view observed route
- > inspect IP/domain intelligence
- > open campaign graph
- > generate PDF + JSON report
- > repeat with phishing + legitimate email

No step requires a local GPU or a fragile external API.

18. Verification sources checked for the stack

Python 3.13.15 - Python.org release page - <https://www.python.org/downloads/release/python-31315/>

Node.js 24.21.0 LTS - Node.js release blog - <https://nodejs.org/en/blog/release>

React 19.3 - React official release - <https://react.dev/blog/2026/09/09/react-19-3>

PyTorch 2.14 - PyTorch release blog - <https://pytorch.org/blog/pytorch-2-14-release-blog/>

Transformers 5.17.0 stable docs - Hugging Face installation -
<https://huggingface.co/docs/transformers/main/installation>

FastAPI 0.141.1 - PyPI - <https://pypi.org/project/fastapi/>

mail-parser 4.6.5 - PyPI - <https://pypi.org/project/mail-parser/>

geoip2 5.3.0 - PyPI - <https://pypi.org/project/geoip2/>

PostgreSQL 18.6 - PostgreSQL versioning/release information - <https://www.postgresql.org/support/versioning/>

Source note: version selections reflect the checked public sources above on 14 September 2026. Package availability can change; always let the final lock files record the exact resolved versions used by the team.

MAILTRACE AI | Team Echoshield | SIH26106